

Why Language Algorithms Are Hard

The application of algorithmic thinking to language goes back at least as far as Descartes, who noticed that although there are infinite numbers, anyone with a rudimentary understanding of arithmetic knows how to create or interpret a number they've never encountered before. For example, maybe you've never encountered the number 14,326—never counted that high, never read a financial report about that many dollars, never mashed exactly those keys on the keyboard. And yet I'm confident that you can easily grasp exactly how high it is, what numbers are higher or lower than it, and how to manipulate it in equations.

The algorithm that lets us easily understand hitherto unimagined numbers is simply a combination of the 10 digits (0–9), memorized in order, and the place system. We know that 14,326 is one higher than 14,325 because the digit 6 comes one after the digit 5 in order, they occupy the same place in their respective numbers, and the digits in all the other places are the same. Knowing the digits and the place system enables us to instantly have an idea of how 14,326 is similar to 14,325 and how both are larger than 12 and smaller than 1,000,000. We can also understand at a glance that 14,326 is similar to 4,326 in some respects but differs greatly in size.

Language is not the same. If you are learning English and you see the word *stage* for the first time, you cannot reliably reason about its meaning simply by noting its similarity to *stale* or *stake* or *state* or *stave* or *stade* or *sage*, even though those words differ from *stage* about as much as 14,326 does from 14,325. Nor can you reliably suppose that a bacterium is larger than an elk because of the number of syllables and characters in the words. Even supposedly reliable rules of language, like adding *s* to form plurals in English, can lead us badly astray when we infer that the word “princes” refers to less of something than the word “princess.”

In order to use algorithms with language, we must either make language simpler, so that the short mathematical algorithms we have explored so far can reliably work with it, or make our algorithms smarter, so that they can deal with the messy complexity of human language as it has developed naturally. We'll do the latter.